Panduan Belajar Mandiri Pertemuan 1: Fondasi Web & Semantik HTML

Oct 25, 2026 · @Raihan

Pengantar

Panduan ini merangkum seluruh materi Pertemuan 1 supaya kamu bisa belajar ulang sendiri, dari cara kerja web sampai membuat halaman profil dengan HTML semantik. Setiap bagian dimulai dari konsep, lalu contoh kode, lalu hal yang sering membuat bingung.

Tujuan pertemuan

- Memahami cara kerja dan arsitektur web.
- Mengetahui perbedaan HTML biasa dan HTML semantik.
- Membuat kerangka halaman profil pertamamu.

Cara memakai panduan

1. Baca satu bagian, lalu ketik ulang contohnya sendiri di VS Code. Jangan hanya copy-paste, karena tanganmu perlu terbiasa menulis tag.
2. Simpan semuanya di satu file `index.html` di dalam folder `Pertemuan_1`. Setiap bagian menambah isi halaman yang sama.
3. Buka file di browser (atau pakai ekstensi Live Server) dan refresh setiap selesai menyimpan.
4. Kerjakan latihan di bagian akhir sebelum membuka kunci jawabannya.

Peta belajar

Langkah	Topik	Yang ditambahkan ke halaman profil
1	Tag dan elemen	Judul dan paragraf
2	Kerangka dokumen	`html`, `head`, `body`, judul tab
3	Nesting dan atribut	Gambar, daftar skill, link
4	Block vs inline	Penekanan kata di perkenalan
5	Semantik HTML	`header`, `nav`, `main`, `section`, `article`, `aside`, `footer`
6	Proyek akhir	Halaman profil lengkap

Alat yang dibutuhkan: Visual Studio Code, Google Chrome, dan ekstensi Live Server di VS Code (opsional, tapi halaman akan refresh otomatis saat disimpan).

Bagaimana web bekerja

Web bekerja dengan pola permintaan dan jawaban: browser meminta, server menjawab dengan file, lalu browser menyusun file itu menjadi halaman.

1. Kamu mengetik alamat (URL). Browser, yang disebut *client*, mengirim *request* ke server.
2. Server membalas dengan file HTML, ditambah CSS, JavaScript, dan gambar bila ada.
3. Browser membaca HTML dan membangun **DOM**, yaitu pohon objek yang berisi semua elemen halaman.
4. Browser menerapkan CSS pada pohon itu lalu menggambarnya di layar. Tahap ini disebut *render*.
5. JavaScript, bila ada, dapat mengubah pohon tadi setelah halaman tampil.

Tiga teknologi dengan tiga peran berbeda:

Teknologi	Peran	Analogi membangun rumah
HTML	Struktur dan makna	Pondasi dan batu bata: menentukan letak kamar dan pintu
CSS	Tampilan	Cat dan dekorasi
JavaScript	Perilaku dan interaksi	Saklar listrik: lampu menyala saat ditekan

Catatan penting: HTML mengatur *makna*, bukan tampilan. Judul yang tampil besar dan teks yang tebal berasal dari CSS bawaan browser, bukan dari HTML itu sendiri. Hal ini akan kamu pahami lebih dalam di bagian Block vs Inline dan Semantik HTML.

HTML, tag, dan elemen

HTML adalah file teks biasa yang berisi label untuk memberi tahu browser: bagian ini adalah apa. Browser tidak tahu mana judul dan mana paragraf kecuali kamu memberi labelnya.

```
<h1>Halo Dunia</h1>
<p>Ini paragraf pertamaku.</p>
```

- Label itu disebut **tag**. Tag punya pembuka `<h1>` dan penutup `</h1>` (ada garis miring).
- Pembuka + isi + penutup = satu **elemen**.

Analogi: tag seperti label di kardus pindahan. Isi kardusnya sama, tetapi label "DAPUR" atau "KAMAR" memberi tahu tukang angkut mau ditaruh di mana.

Fungsi tag

Tag memberi label jenis atau makna pada isinya, supaya browser tahu isi itu apa.

Tag	Artinya
`<h1>`	Judul utama
`<p>`	Paragraf
`<a>`	Link
`<img>`	Gambar

Heading `h1` sampai `h6`

Heading membentuk hierarki: `h1` judul utama, `h2` sub-judul, `h3` sub-sub-judul, dan seterusnya. Hierarki hanya berlaku untuk heading. Tag `<p>` dan `<a>` tidak punya hierarki, tetapi tetap punya label.

Teks `<h2>` tampak lebih kecil dari `<h1>` hanya karena tampilan bawaan browser. Arti sebenarnya: `h2` adalah sub-judul level 2. Dengan CSS kamu bisa membuat `h2` lebih besar dari `h1`, dan maknanya tetap sub-judul.

Prinsip yang harus diingat: HTML mengatur makna, bukan tampilan.

Praktik

1. Buat folder `Pertemuan_1`, lalu file `index.html` di dalamnya.
2. Isi dengan `<h1>` dan `<p>` seperti contoh di atas.
3. Double-click file itu supaya terbuka di browser.
4. Ubah `<h1>` menjadi `<h2>`, simpan, refresh. Perhatikan apa yang berubah.

Kerangka dokumen HTML

Halaman HTML yang lengkap punya kerangka tetap. Contoh dua baris di bagian sebelumnya sudah tampil di browser, tetapi belum lengkap.

```
<!DOCTYPE html>
<html lang="id">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Profil Saya</title>
  </head>
  <body>
    <h1>Halo, saya [Namamu]</h1>
    <p>Perkenalan singkat.</p>
  </body>
</html>
```

Bagian	Fungsi
`<!DOCTYPE html>`	Memberi tahu browser bahwa ini HTML modern. Selalu baris pertama.
`<html lang="id">`	Pembungkus seluruh halaman. `lang="id"` menandai bahasa Indonesia, membantu browser dan screen reader.
`<head>`	Info tentang halaman. Tidak tampil sebagai isi di layar.
`<body>`	Semua isi yang tampil di layar.
`<title>`	Teks di tab browser, bookmark, dan judul di hasil pencarian. Letaknya di dalam `head`.
`<meta charset="UTF-8">`	Agar huruf dan simbol (é, emoji, dll.) tampil benar.
`<meta name="viewport" ...>`	Agar halaman pas di layar HP (lihat di bawah).

Analogi buku: `head` adalah sampul dan data buku (judul, penulis), `body` adalah halaman-halaman isinya.

Pintasan di VS Code: ketik `!` lalu tekan Tab, dan kerangka di atas muncul otomatis.

Apa isi `head` sebenarnya

`head` bukan hanya untuk SEO. Metadata di dalamnya melayani tiga pihak:

- **Browser**: `charset` agar huruf benar, `viewport` agar pas di HP, dan link ke file CSS.
- **Pengguna**: `<title>` muncul di tab, bookmark, dan hasil pencarian.
- **Mesin pencari**: deskripsi dan info halaman.

Meta viewport

Tanpa baris `viewport`, browser HP berpura-pura layarnya selebar sekitar 980px, lalu mengecilkan seluruh halaman sampai teks sulit dibaca. Dengan baris ini, browser memakai lebar asli layar perangkat.

- `width=device-width`: lebar halaman sama dengan lebar layar perangkat.
- `initial-scale=1.0`: zoom awal 100%.

Efeknya baru terasa jelas saat belajar CSS responsif, tetapi baris ini wajib ada sejak awal.

Eksperimen: `<h1>` di dalam `<head>`

Pindahkan `<h1>` ke dalam `<head>`, simpan, refresh, lalu buka F12 (Elements). Hasilnya: `<h1>` tetap tampil, dan di Elements posisinya sudah berada di `body`.

Sebabnya: `head` hanya boleh berisi elemen metadata (`meta`, `title`, `link`, dan sejenisnya). Saat parser bertemu `<h1>`, ia tahu elemen itu hanya valid di `body`, sehingga otomatis menutup `head` dan membuka `body`.

Pelajaran penting: browser tidak menampilkan error saat HTML-mu salah. Ia diam-diam memperbaikinya. Enaknya halaman tidak crash, bahayanya hasilnya bisa berbeda dari maksudmu. Karena itu kerapian struktur penting.

Nesting dan atribut

Nesting: elemen di dalam elemen

Elemen boleh berisi elemen lain. Chrome mengubah teks HTML menjadi **pohon** (DOM):

```
<body>
  <p>Aku <strong>suka</strong> coding.</p>
</body>
```

```
body
└── p
    ├── "Aku "
    ├── strong
    │   └── "suka"
    └── " coding."
```

- **Parent**: elemen yang membungkus (`p` adalah parent dari `strong`).
- **Child**: elemen di dalamnya (`strong` adalah child dari `p`).
- **Sibling**: elemen yang sejajar di bawah parent yang sama.

Aturan penutup: tutup dari yang paling dalam dulu, seperti kotak di dalam kotak.

```
<p><strong>Halo</strong></p>   <!-- benar -->
<p><strong>Halo</p></strong>   <!-- salah: saling tumpang tindih -->
```

Yang salah tidak membuat error, tetapi Chrome menebak-nebak perbaikannya, dan hasil pohonnya bisa berbeda dari maksudmu. Indentasi bukan syarat browser, tetapi membantumu melihat pohonnya.

Ctrl+U vs tab Elements

- **Ctrl+U** (View Source) menampilkan teks HTML asli yang kamu tulis.
- **F12, tab Elements** menampilkan DOM, yaitu pohon setelah diproses dan diperbaiki Chrome.

Keduanya bisa berbeda, misalnya pada eksperimen `<h1>` di dalam `<head>` tadi. Halaman yang kamu lihat adalah gambar dari DOM, dan JavaScript nanti mengubah DOM, bukan file teksnya.

Atribut

Atribut adalah info tambahan di tag pembuka, bentuknya `nama="nilai"`.

```
<a href="https://github.com/zennthh" target="_blank">github</a>
```

Di sini `a` adalah tag, `href` dan `target` adalah nama atribut, sedangkan `"https://..."` dan `"_blank"` adalah nilainya.

- Atribut hanya ditulis di tag pembuka, tidak pernah di tag penutup.
- Nilai diberi tanda kutip, urutan atribut bebas, dipisah spasi.

Atribut	Dipakai di	Yang dilakukan browser
`href`	`<a>`	Saat diklik, pindah ke alamat itu
`target="_blank"`	`<a>`	Membuka di tab baru
`src`	`<img>`	Mengambil file gambar dari lokasi itu
`alt`	`<img>`	Teks pengganti gambar, dibacakan screen reader
`id`	Semua tag	Nama unik elemen (target link, nanti dipakai CSS dan JS)
`class`	Semua tag	Nama kelompok (nanti dipakai CSS dan JS)

Path file di `href` dan `src`

Penulisan	Artinya
`foto.jpg`	File di folder yang sama dengan `index.html`
`img/foto.jpg`	Masuk ke subfolder `img`
`../foto.jpg`	Naik satu folder ke atas
`/img/foto.jpg`	Dari akar situs, bukan dari folder ini (sumber bug umum)
`https://situs.com/foto.jpg`	Alamat lengkap di internet

Penyebab nomor satu "gambar tidak muncul" adalah path yang salah. Periksa path dulu.

Link ke dalam halaman (anchor)

```
<a href="#kontak">Ke Kontak</a>
...
<h2 id="kontak">Kontak Saya</h2>
```

`#kontak` berarti "cari elemen ber-`id="kontak"` di halaman ini lalu gulir ke sana". Kalau id-nya tidak ada atau beda satu huruf, tidak terjadi apa-apa.

List: `ul`, `ol`, `li`

```
<ul>            <!-- unordered: bullet, urutan tidak penting -->
  <li>HTML</li> <!-- satu item -->
  <li>CSS</li>
</ul>

<ol>            <!-- ordered: bernomor, urutan penting -->
  <li>Buka VS Code</li>
  <li>Buat file</li>
</ol>
```

`<li>` hanya boleh langsung berada di dalam `<ul>` atau `<ol>`. Pakai list, bukan `<p>` berulang, karena list membawa makna "ini daftar": screen reader mengumumkan "daftar, 3 item".

Gambar: `<img>`

```
<img src="foto-profil.jpg" alt="Foto saya sedang duduk di depan laptop">
```

Saat Chrome bertemu `<img>`, ia mengirim request terpisah untuk mengambil file di `src`, tanpa menunggu: sisa HTML tetap dibaca sambil gambar diunduh. Itu sebabnya pada koneksi lambat teks muncul lebih dulu. Kamu bisa melihat setiap request di F12, tab Network, lalu refresh.

`<img>` tidak punya tag penutup karena tidak ada isi yang perlu dibungkus, semua informasinya ada di atribut. Tag seperti ini disebut *void element* (`<img>`, `<br>`, `<meta>`, `<input>`).

`alt` bukan formalitas. Ia dipakai saat gambar gagal dimuat, saat screen reader membacakan gambar, dan saat mesin pencari memahami isi gambar. Tulis deskripsi yang bermakna, bukan "gambar1".

Praktik

Tambahkan ke `index.html` profilmu: satu `<img>` di bawah `<h1>`, daftar `<ul>` berisi tiga skill, dan link GitHub dengan `target="_blank"`. Lalu coba: ubah `src` menjadi nama file yang salah dan lihat di mana `alt` muncul, buka F12, tab Network, dan temukan request gambarmu.

Block vs inline

Setiap elemen punya aturan tentang bagaimana ia menempati ruang di halaman. Ini fondasi untuk CSS (box-model di Pertemuan 2) dan kunci memahami kenapa tata letak kadang "berperilaku aneh".

	Block	Inline
Posisi	Mulai di baris baru	Mengalir di dalam baris, di samping teks
Lebar	Selebar ruang yang tersedia di parent-nya	Hanya selebar isinya
Bisa diatur `width` dan `height`	Ya	Tidak
Contoh	`h1` - `h6`, `p`, `div`, `ul`, `ol`, `li`, `header`, `nav`, `main`, `section`, `article`, `aside`, `footer`	`a`, `span`, `strong`, `em`, `img`

Analogi: block seperti paragraf di buku, selalu mulai baris baru dan memenuhi lebar halaman. Inline seperti kata yang distabilo di dalam kalimat, tetap di tempatnya dan selebar katanya.

`<img>` mengejutkan karena termasuk inline. Itu sebabnya dua gambar berurutan tampil berdampingan, bukan bertumpuk. Semua tag semantik di bagian berikutnya adalah block.

Dari mana perilaku ini berasal

Block dan inline **bukan bawaan HTML**, melainkan properti CSS bernama `display`. Chrome punya stylesheet bawaan (*user-agent stylesheet*) yang memberi nilai default: `p`, `h1`, dan `div` bernilai `display: block`, sedangkan `a`, `span`, dan `strong` bernilai `display: inline`.

Jadi HTML hanya memberi makna, sedangkan perilaku block dan inline adalah aturan CSS bawaan browser yang bisa kamu ubah nanti. Itu juga sebabnya web "polos" tanpa CSS tetap punya tampilan (heading besar, teks tebal, bullet).

Satu nilai lagi untuk dikenal: `inline-block`, yaitu berdampingan seperti inline tetapi bisa diatur lebar dan tingginya seperti block. Ini dibahas di CSS.

Aturan nesting yang lahir dari sini

- Inline boleh berada di dalam block: `<p>Aku <strong>suka</strong> coding</p>`.
- `<p>` hanya boleh berisi konten inline. Kalau kamu menaruh `<h2>` atau `<div>` di dalam `<p>`, parser Chrome otomatis menutup `<p>` lebih awal. Hasilnya `<h2>` keluar dari `<p>`, dan `</p>` yang tersisa membuat `<p>` kosong di bawahnya.

Whitespace di HTML

Chrome menggabungkan spasi dan Enter berlebih menjadi satu spasi saat merender. Menulis lima spasi berturut-turut tetap tampil sebagai satu spasi. DOM tetap menyimpan teks aslinya, pemadatan terjadi saat render. Untuk pindah baris di dalam teks pakai `<br>`, bukan menekan Enter berkali-kali.

Tag inline yang berguna

Tag	Makna	Tampilan bawaan
`<strong>`	Teks penting	Tebal
`<em>`	Penekanan	Miring
`<span>`	Tanpa makna, wadah inline untuk styling CSS nanti	Tidak berubah
` `	Pindah baris (void element)	-

`<strong>` dan `<em>` membawa makna (dibaca screen reader dengan penekanan), berbeda dengan sekadar membuat teks tebal secara visual.

Praktik

Ubah perkenalanmu menjadi:

```
<p>Halo, aku <strong>[Namamu]</strong>, mahasiswa <span>informatika</span>
yang sedang belajar <em>web development</em>.</p>
```

Lalu coba:

1. Tambahkan `<img>` kedua tepat setelah gambar pertama. Berdampingan atau bertumpuk?
2. Tambahkan lima spasi dan tiga Enter di tengah kalimat. Apa yang tampil?
3. Taruh `<h2>Tes</h2>` di dalam `<p>`, refresh, lalu lihat F12, tab Elements. Kembalikan setelah selesai.
4. Di Elements, arahkan kursor ke `<p>` dan `<strong>`: bandingkan bentuk sorotannya. Klik elemen, buka tab Computed, cari `display`.

Semantik HTML

HTML semantik berarti memakai tag yang **bermakna** (`<header>`, `<nav>`, `<main>`), bukan `<div>` untuk segalanya. Dua halaman bisa tampil nyaris sama, tetapi yang satu bisa dipahami mesin dan yang lain hanya kotak kosong.

```
<!-- Cara lama: semua ruang bernama div -->
<div class="header">...</div>
<div class="nav">...</div>

<!-- Semantik: nama ruang jelas -->
<header>...</header>
<nav>...</nav>
```

Kenapa tidak `<div>` semua?

Analogi dokumen Word. Kamu bisa membuat judul bab dengan memblok teks, memperbesar, dan menebalkannya manual, atau dengan Heading Style. Tampilannya sama, tetapi hanya Heading Style yang membuat daftar isi otomatis dan panel navigasi bekerja. `<div>` itu cara manual, tag semantik itu Heading Style.

`class="header"` tidak cukup karena `class` hanyalah nama bebas buatan penulis (bisa `header`, `kepala`, atau `xyz123`). `<header>` adalah bagian dari kosakata standar yang disepakati semua browser dan alat.

Siapa yang membaca makna itu:

- **Screen reader.** Chrome mengubah DOM menjadi *accessibility tree*. Tag semantik menjadi landmark, sehingga pengguna tunanetra bisa melompat langsung ke navigasi atau konten utama tanpa mendengar seluruh halaman dari atas.
- **Browser.** Tag semantik membawa perilaku bawaan. Contoh: `<button>` bisa dijangkau dengan tombol Tab dan dijalankan dengan Enter, sedangkan `<div onclick="...">` tidak bisa dijangkau keyboard sama sekali.
- **Mesin pencari dan mode baca.** Struktur membantu membedakan konten inti dari menu dan sidebar. Semantik membantu mesin memahami halaman, tetapi bukan tombol ajaib untuk naik peringkat.
- **Programmer lain.** Deretan `<div><div><div>` (disebut *div soup*) sulit dibaca dan dirawat, sedangkan `<header><nav>` langsung jelas.

Tampilan di layar tidak berubah. Perbedaannya murni pada makna.

Accessibility tree: lihat sendiri di Chrome

Tag	Role di accessibility tree
`<header>`	banner (bila di level halaman)
`<nav>`	navigation
`<main>`	main
`<aside>`	complementary
`<footer>`	contentinfo (bila di level halaman)
`<article>`	article
`<section>`	region (bila punya heading)
`<div>`	generic (tanpa makna)

Cara melihatnya: F12, tab Elements, pilih sebuah tag, lalu buka panel Accessibility dan baca **Role**-nya. Coba ubah `<nav>` menjadi `<div>` sementara dan bandingkan role-nya.

Tujuh tag semantik dan tes penentunya

Tag	Tanyakan	Aturan
`<header>`	Ini pengantar atau kepala?	Boleh juga di dalam `article` atau `section` sebagai kepala bagian itu

Tag	Tanyakan	Aturan
`<nav>`	Ini blok link navigasi utama?	Jangan untuk sembarang kumpulan link
`<main>`	Ini konten inti halaman?	Hanya satu per halaman, bukan di dalam header, footer, atau nav
`<section>`	Ini bagian bertema yang punya heading?	Tanpa heading biasanya salah pilih tag
`<article>`	Kalau dipotong dan ditaruh di tempat lain, masih berdiri sendiri?	Kartu proyek, post blog, komentar
`<aside>`	Ini pelengkap, bukan inti?	Sidebar, tautan terkait
`<footer>`	Ini penutup atau info tambahan?	Copyright, kontak

Alur memilih: berdiri sendiri, pakai `article`. Bagian bertema yang berjudul, pakai `section`. Hanya wadah untuk styling tanpa makna, pakai `div`. `<div>` tidak dilarang, hanya menjadi pilihan terakhir setelah tag bermakna.

`<section>`, `id`, dan link menu

Tiga hal ini sering tertukar:

	Peran
`<section>`	**Makna**: mengelompokkan satu topik
`id="skill"`	**Alamat**: nama unik supaya bisa dituju
`href="#skill"`	**Penunjuk**: menuju alamat itu

Yang membuat link menu menggulir ke bagian yang benar adalah atribut `id`, bukan tag `<section>`. Tag apa pun yang punya `id` bisa menjadi tujuan.

`<aside>` secara khusus

`<aside>` adalah konten pelengkap: berkaitan dengan halaman, tetapi bukan bagian isi intinya. Tes penentu: **kalau bagian ini dihapus, halaman masih lengkap? Kalau ya, itu `aside`.**

Cocok pakai `<aside>`	Tidak cocok
"Artikel Terkait"	Isi artikel itu sendiri

Cocok pakai `<aside>`	Tidak cocok
Kotak kutipan atau catatan tambahan	Daftar skill di profil (itu inti)
Info penulis, tautan pilihan	Menu utama (itu `nav`)

Kesalahan paling umum: mengira `aside` sama dengan "sidebar" dan memakainya untuk apa pun yang letaknya di samping. Letak itu urusan CSS, makna itu urusan HTML. `aside` melengkapi induk terdekatnya, sehingga ada dua penempatan yang valid:

- Di luar `<main>`, sejajar dengan `header` dan `footer`: pelengkap untuk seluruh halaman.
- Di dalam `<article>`: pelengkap untuk artikel itu saja.

Isi `aside` jangan menduplikasi inti. Link GitHub yang sudah ada di "Kontak" tidak perlu diulang di `aside`.

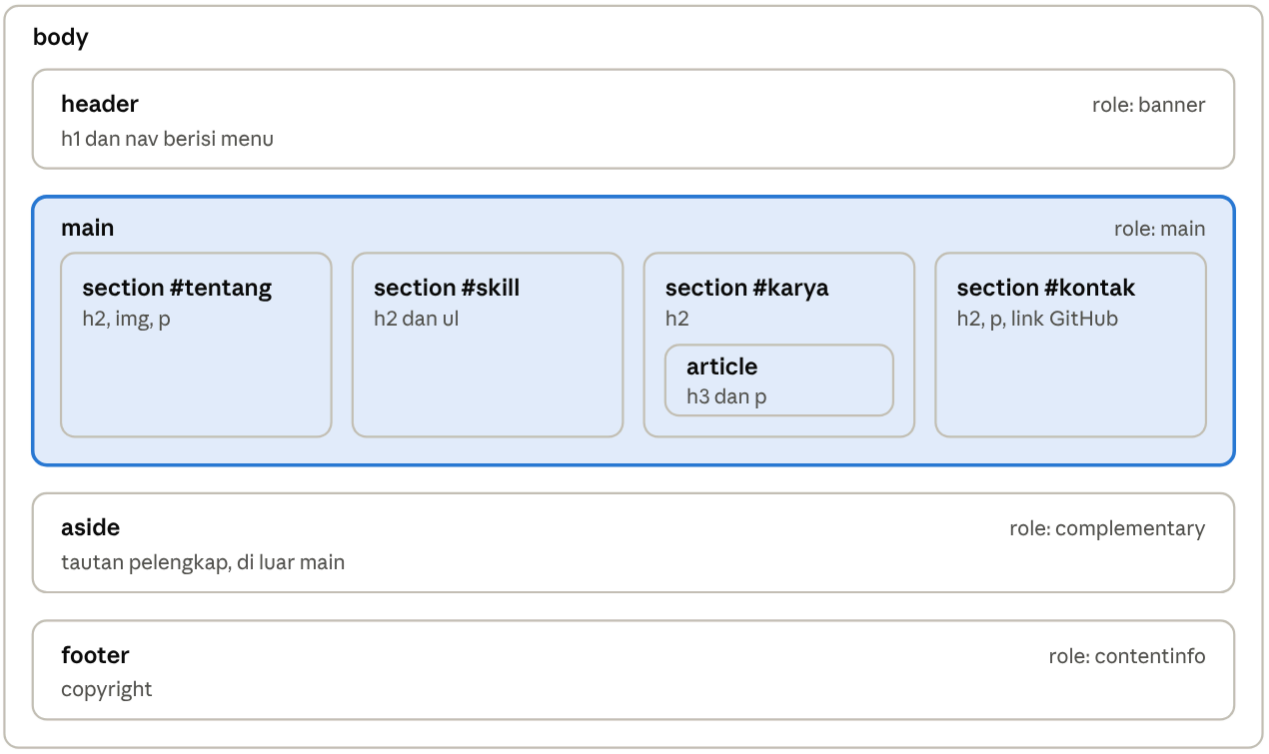
Aturan heading

- Satu `<h1>` per halaman, berisi judul halaman.
- Turun berurutan: `h1`, lalu `h2`, lalu `h3`. Jangan melompat, dan jangan memilih heading karena ukurannya (ukuran adalah urusan CSS).
- Heading membentuk outline. Bagian yang sejajar memakai level yang sama. Contoh: "Kontak Saya" sejajar dengan "Skill Saya", jadi keduanya `h2`. Kalau "Kontak" memakai `h3`, artinya ia sub-bagian dari Skill, dan itu salah.

Proyek akhir: halaman profil lengkap

Ini hasil akhir Pertemuan 1. Semua bagian sebelumnya bertemu di sini. Salin strukturnya, lalu ganti isinya dengan datamu sendiri (nama, foto, skill, proyek, link).

Konten inti hidup di satu main, tag semantik lain mengapitnya



struktur halaman profil · 7 tag semantik di dalam body

Setiap kotak adalah satu tag, dan semuanya berjenis block sehingga tersusun dari atas ke bawah. Role di sisi kanan adalah yang dibaca screen reader. Kode lengkapnya ada di bawah.

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Profil [Namamu]</title>
</head>
<body>

  <header>
    <h1>Halo, saya [Namamu]</h1>
    <nav>
      <ul>
        <li><a href="#tentang">Tentang</a></li>
        <li><a href="#skill">Skill</a></li>
        <li><a href="#karya">Karya</a></li>
        <li><a href="#kontak">Kontak</a></li>
      </ul>
    </nav>
```

```
  </header>

  <main>
    <section id="tentang">
      <h2>Tentang Saya</h2>
      <img src="foto-profil.jpg" alt="[Deskripsi isi fotomu]">
      <p>
        Saya mahasiswa <strong>[jurusan]</strong>
        yang sedang belajar <em>web development</em>.
      </p>
    </section>

    <section id="skill">
      <h2>Skill Saya</h2>
      <ul>
        <li>HTML</li>
        <li>CSS</li>
        <li>JavaScript</li>
      </ul>
    </section>

    <section id="karya">
      <h2>Karya</h2>
      <article>
        <h3>[Judul proyek]</h3>
        <p>[Deskripsi singkat proyek.]</p>
      </article>
    </section>

    <section id="kontak">
      <h2>Kontak Saya</h2>
      <p>
        Kunjungi
        <a href="https://github.com/[username]" target="_blank">profil GitHub
saya</a>.
      </p>
    </section>
  </main>

  <aside>
    <h2>Sumber Belajar</h2>
    <ul>
      <li><a href="https://developer.mozilla.org" target="_blank">MDN Web
Docs</a></li>
      <li><a href="https://validator.w3.org" target="_blank">W3C
Validator</a></li>
```

```
    </ul>
  </aside>

  <footer>
    <p>&copy; 2026 [Namamu]</p>
  </footer>

</body>
</html>
```

Hal yang sengaja dijaga di struktur ini

- **Heading berurutan:** satu `h1`, bagian utama `h2`, judul di dalam `article` memakai `h3`. Heading `aside` memakai `h2` karena sejajar dengan section.
- **Anchor cocok:** setiap `href="#x"` punya `id="x"`.
- **`main` hanya satu**, dan `aside` berada di luarnya.
- **`alt` deskriptif**, dan teks link menjelaskan tujuan ("profil GitHub saya", bukan "klik di sini").
- **Belum ada link ke `style.css`**, karena filenya belum ada. Link itu ditambahkan di `<head>` pada Pertemuan 2.

Uji di Chrome

1. Klik tiap menu: semuanya harus menggulir ke bagian yang benar.
2. F12, tab Elements, panel Accessibility: periksa role `banner`, `navigation`, `main`, `complementary`, dan `contentinfo`.
3. Tempel kodenya ke validator.w3.org (tab "Direct Input"): seharusnya nol error.
4. Ctrl+U dan tab Elements: bandingkan, apakah ada yang berbeda?

Pertanyaan yang sering muncul

Kenapa tidak `<div>` semua, toh tampilannya sama? Mesin (screen reader, mesin pencari, mode baca) tidak bisa membaca maksud `<div>`. Tag semantik memberi tahu mesin bagian mana yang menu, konten utama, dan penutup. Sama seperti Heading Style di Word: tampilannya mirip teks tebal biasa, tetapi daftar isi hanya bisa jalan kalau memakai Heading.

Apakah semantik membuat web naik peringkat di Google? Tidak otomatis. Semantik membantu mesin pencari memahami struktur halaman, tetapi bukan tombol ajaib untuk peringkat. Manfaat terbesarnya adalah aksesibilitas dan kerapian kode.

Apa beda `section`, `article`, dan `div`? Berdiri sendiri (kalau dipotong dan ditaruh di tempat lain tetap masuk akal): `article`. Bagian bertema yang punya heading: `section`. Hanya wadah untuk styling tanpa makna: `div`.

Boleh pakai `<main>` dua kali? Tidak. Hanya satu `<main>` per halaman, dan tidak boleh berada di dalam `header`, `footer`, atau `nav`.

Apa beda `aside` dan `section`? `section` adalah bagian dari isi inti. `aside` adalah tambahan di luar inti. Tes: kalau dihapus, halaman masih lengkap? Kalau ya, itu `aside`.

Apakah `aside` harus berada di samping? Tidak. Posisi diatur CSS, bukan HTML. Hari ini tampilannya masih menumpuk ke bawah, dan itu normal.

Apa beda `id` dan `class`? `id` unik, hanya boleh dipakai satu elemen dalam satu halaman. `class` boleh dipakai banyak elemen. Keduanya dipakai oleh CSS dan JavaScript nanti.

Kenapa webku terlihat jelek dan polos? Belum diberi styling. HTML hanya mengatur struktur dan makna, tampilannya diatur CSS di Pertemuan 2.

Kenapa link menu di halamanku tidak berpindah ke bagian yang dituju? Periksa `id`. `href="#kontak"` hanya bekerja kalau ada elemen dengan `id="kontak"` yang ditulis persis sama. Satu huruf berbeda membuatnya diam saja.

Kenapa gambarku tidak muncul? Penyebab paling umum adalah path yang salah. Periksa nama file (termasuk huruf besar-kecil dan ekstensi) dan apakah file ada di folder yang benar. Teks `alt` akan tampil sebagai pengganti.

Apa beda `src="foto.jpg"` dan `src="img/foto.jpg"`? Yang pertama mencari file di folder yang sama dengan `index.html`. Yang kedua mencari di dalam subfolder `img`.

Kenapa `<h1>` yang kutaruh di `<head>` pindah ke `<body>`? Browser tahu `head` hanya boleh berisi metadata. Saat bertemu elemen konten, ia otomatis menutup `head` dan membuka `body`. Browser memperbaiki HTML yang salah secara diam-diam, bukan menampilkan error.

Kenapa muncul `<p>` kosong setelah aku menaruh `<h2>` di dalam `<p>`? `<p>` hanya boleh berisi konten inline. Saat parser bertemu `<h2>` (block), ia menutup `<p>` lebih awal. Tag `</p>` yang tersisa lalu membuat `<p>` baru yang kosong.

Kenapa lima spasi di HTML hanya tampil satu spasi? Browser menggabungkan spasi dan Enter berlebih menjadi satu spasi saat render. Pakai `<br>` untuk pindah baris.

Apa bedanya Ctrl+U dan tab Elements? Ctrl+U menampilkan teks HTML asli yang kamu tulis. Elements menampilkan DOM, yaitu pohon setelah diproses dan diperbaiki Chrome. Keduanya bisa berbeda.

Kenapa gambar tampil berdampingan, padahal aku menaruhnya di baris berbeda? `<img>` termasuk elemen inline, jadi mengalir seperti teks. Elemen block seperti `<p>` yang memulai baris baru.

Kenapa `<img>` tidak punya tag penutup? Tidak ada isi yang perlu dibungkus, semua informasi ada di atribut. Tag seperti ini disebut void element.

Apakah `<div>` dilarang? Tidak. `<div>` tetap berguna sebagai wadah netral, misalnya untuk layout CSS. Aturannya: pakai tag bermakna dulu, `div` sebagai pilihan terakhir.

Kenapa `lang="id"` penting? Browser dan screen reader memakainya untuk memilih bahasa pengucapan dan menawarkan terjemahan yang benar.

Latihan mandiri

Jawab dengan kata sendiri sebelum membuka kunci jawaban.

1. Apa fungsi tag? Pada `<p>Halo</p>`, mana tag pembuka dan mana penutup?
2. Tag apa untuk sub-judul "Kontak Saya" di bawah judul utama, dan kenapa bukan `<p>`?
3. Apa beda isi `<head>` dan `<body>`? Di mana teks `<title>` muncul?
4. Pada `<a href="x" target="_blank">`, mana nama atribut dan mana nilainya?
5. Mana yang benar: `<ul><li>A</ul></li>` atau `<ul><li>A</li></ul>`? Kenapa?
6. Apa beda Ctrl+U dan tab Elements di Chrome?
7. `<img>` termasuk block atau inline? Bagaimana membuktikannya di halamanmu?
8. Apa yang terjadi saat `<h2>` ditaruh di dalam `<p>`, dan kenapa?
9. Tentukan tag paling tepat dan alasannya: (a) kartu proyek, (b) bagian "Skill Saya", (c) daftar tautan terkait di samping.
10. Kenapa `href="#kontak"` butuh `id="kontak"`? Apa yang terjadi kalau tidak ada?
11. Kenapa tidak boleh lompat dari `h1` ke `h3`? Level apa untuk judul proyek di dalam `article` di dalam section "Karya"?
12. Kenapa `<header>` lebih berguna daripada `<div class="header">` padahal tampilannya identik? Sebut minimal dua pihak yang diuntungkan.

Latihan perbaiki kode

Kode berikut punya sedikitnya delapan masalah. Temukan dan perbaiki semuanya.

```
<html>
<head>
  <title>Profil</title>
  <h1>Selammat datang</h1>
</head>
<body>
  <h1>Halo</h1>
  <img src="foto.jpg">
  <h3>Kontak</h3>
  <ul><li>HTML</ul></li>
  <a href="#kontak">Menu</a>
</body>
</html>
```

Kunci jawaban

1. Tag memberi label jenis atau makna pada isinya. Pembuka `<p>` (tanpa garis miring), penutup `</p>` (dengan garis miring).
2. `<h2>`, karena itu sub-judul dan browser serta mesin perlu tahu itu judul, bukan paragraf biasa.
3. `head` berisi metadata (info untuk browser, pengguna, dan mesin pencari) yang tidak tampil sebagai isi. `body` berisi isi yang tampil di layar. `title` muncul di tab browser, bookmark, dan hasil pencarian.
4. Nama atribut: `href` dan `target`. Nilai: `"x"` dan `"_blank"`.
5. `<ul><li>A</li></ul>`. Tag harus ditutup dari yang paling dalam dulu, dan `li` harus langsung berada di dalam `ul` atau `ol`.
6. Ctrl+U menampilkan teks HTML asli. Elements menampilkan DOM setelah diproses dan diperbaiki Chrome.
7. Inline. Taruh dua `<img>` berurutan: tampil berdampingan. Di Elements, tab Computed menunjukkan `display: inline`.
8. Browser menutup `<p>` lebih awal karena `<p>` hanya boleh berisi konten inline. `<h2>` keluar dari `<p>`, dan `</p>` yang tersisa membuat `<p>` kosong.
9. (a) `<article>`, karena berdiri sendiri. (b) `<section>`, bagian bertema yang berjudul. (c) `<aside>`, karena pelengkap, bukan inti.
10. Link `#kontak` mencari elemen ber-`id="kontak"` lalu menggulir ke sana. Kalau id tidak ada atau beda, tidak terjadi apa-apa.
11. Heading membentuk outline berurutan, melompat membuat struktur tidak masuk akal bagi screen reader. Judul proyek memakai `h3`: `h1` judul halaman, `h2` untuk section "Karya", `h3` untuk judul di dalam `article`.

12. Tampilannya identik, tetapi `<header>` berarti "kepala halaman" bagi mesin. Yang diuntungkan: pengguna screen reader (bisa melompat ke landmark), mesin pencari (memahami struktur), programmer lain (kode lebih mudah dibaca).

Kunci latihan perbaiki kode:

- Tidak ada `<!DOCTYPE html>`.
- Tidak ada `lang="id"` pada `<html>`.
- Tidak ada `<meta charset>` dan `<meta name="viewport">`.
- `<h1>` berada di dalam `<head>`; hapus.
- Typo "Selammat" (kalau `h1` dipindah ke body: "Selamat").
- Ada dua `<h1>`; seharusnya satu per halaman.
- `<img>` tidak punya `alt`.
- `<h3>Kontak</h3>` melompat dari `h1` ke `h3`; seharusnya `h2`.
- `<ul><li>HTML</ul></li>` penutupnya terbalik; seharusnya `</li></ul>`.
- `href="#kontak"` tidak punya elemen ber-`id="kontak"` sebagai tujuan.
- Tidak ada `<main>`, `<header>`, dan `<footer>`; strukturnya belum semantik.

Contekan cepat

Tag	Fungsi	Jenis
`<h1>` - `<h6>`	Judul dan sub-judul berurutan	Block
`<p>`	Paragraf	Block
`<ul>`, `<ol>`, `<li>`	Daftar tak berurut, berurut, dan item	Block
`<a href>`	Link	Inline
`<img src alt>`	Gambar (void element)	Inline
`<strong>`, `<em>`	Teks penting, penekanan	Inline
`<span>`	Wadah inline tanpa makna	Inline
` `	Pindah baris (void element)	-
`<div>`	Wadah netral tanpa makna	Block
`<header>`	Kepala halaman atau bagian	Block
`<nav>`	Navigasi utama	Block
`<main>`	Konten inti (satu per halaman)	Block

Tag	Fungsi	Jenis
`<section>`	Bagian bertema yang berjudul	Block
`<article>`	Konten yang berdiri sendiri	Block
`<aside>`	Pelengkap di luar inti	Block
`<footer>`	Penutup halaman atau bagian	Block

Lima prinsip yang harus diingat:

1. HTML mengatur makna, bukan tampilan.
2. Browser memperbaiki HTML yang salah secara diam-diam, jadi tulis dengan rapi.
3. Tutup tag dari yang paling dalam dulu.
4. Pakai tag bermakna dulu, `<div>` sebagai pilihan terakhir.
5. Heading berurutan, satu `<h1>` per halaman.